Sources

Select all sources

20 Backend Project Ideas : Beginner to Pro - GeeksforGeeks
20 Backend Project Ideas to take you from Beginner to Pro - Developer Roadmaps
A Day in the Life of a Cybersecurity Professional - St. John's University
A Day in the Life of a Random C# Backend Engineer - Just Some Code
A day in the life of ... a penetration tester | Thomas Murray
Backend Development in 2026: The Ultimate Guide for Future Backend Engineers
Build a Cybersecurity Home Lab in 2026: A Beginner Setup That Actually Works
Comprehensive Guide to Cybersecurity Career Paths: Defensive, Offensive, GRC, and Beyond
Computer security - Wikipedia
Cybersecurity - Simple English Wikipedia, the free encyclopedia
Cybersecurity Practice Projects For Beginners - Zero To Mastery
Here are 8 beginner cybersecurity projects you can try this weekend
How to Become a Backend Developer in 2026: Complete Roadmap
How to Build an API | Postman Blog
Try These Cybersecurity Projects For Beginners to Get HIRED in 2025: Top 5 Beginner Cyber Projects
Two Paths to Cybersecurity: Technical vs. Management Roles
WebDevStudios Day in the Life of a Backend Engineer
What Does a Back-End Developer Do? - Coursera
What Does a SOC Analyst Actually Do All Day? - Training Camp
devclub-iitd/summer-of-code-2023: 4-week long hackathon across 4 tracks App, Backend, Frontend and Machine Learning. - GitHub
the complete back end developer roadmap - Code with Mosh

Chat

Cybersecurity Career Guide: Professional Roles and Practical Projects
21 sources·Apr 21, 2026

These sources provide a comprehensive professional guide for aspiring technology specialists, focusing specifically on the fields of backend development and cybersecurity. They outline structured learning roadmapsand educational resources, such as bootcamps and certifications, necessary to achieve job-readiness in 2026. Readers can explore diverse hands-on projects, including building APIs, keyloggers, and security home labs, to strengthen their technical portfolios. Additionally, the texts offer a realistic look at daily responsibilities for roles like cybersecurity analysts, penetration testers, and SOC analysts. By detailing industry trends, salary expectations, and essential toolsets, the sources serve as a practical toolkit for career advancement.

What does a daily schedule look like in cybersecurity?
Which backend projects are best for a beginner portfolio?
How do I build a secure cybersecurity home lab?

Tuesday, April 21

Discuss what these sources say about Risk analysis, in the larger context of High-level Responsibilities.

In the field of cybersecurity, **risk analysis** is categorized as a core **high-level responsibility**, sitting alongside duties such as developing security strategies, setting security standards, and coordinating with various departments. It is a strategic function that moves beyond daily technical tasks like system monitoring or patching to focus on the broader security posture of an organization.

Strategic and Managerial Context

Risk analysis is a defining feature of the **management career track**, where professionals take a big-picture view of an organization's security. In these roles, responsibilities include:

- **Overseeing Risk Strategy:** Leaders and managers are responsible for shaping the risk strategy and aligning it with overarching business goals.
- **Executive Leadership:** High-level positions, such as the **Chief Information Security Officer (CISO)**, require extensive experience in risk management to develop strategic security plans.
- **Security Architecture:** Security architects use **risk assessment** as a primary factor in determining which security controls to standardize and apply within a unified security design.

Specialized Risk Functions

Beyond general management, risk analysis is the central focus of specialized domains like **Governance, Risk, and Compliance (GRC)** and dedicated **Risk Management** tracks:

- **Identification and Evaluation:** Professionals in these fields are tasked with identifying, evaluating, and prioritizing risks that could impact an organization’s data and information systems. This involves assessing both the **likelihood** and the **potential impact** of various threats, such as ransomware or data breaches.
- **Mitigation and Control:** Once risks are identified, risk managers develop strategic plans to minimize and control them, ensuring the organization remains resilient.
- **Compliance Alignment:** GRC experts ensure that risk mitigation strategies and security policies align with industry standards, regulations (such as GDPR or HIPAA), and internal laws.
- **Continuous Monitoring:** A high-level responsibility within risk management is the continuous monitoring of the risk environment to adapt strategies as new and evolving threats emerge.

Skills Required for High-Level Risk Responsibilities

Successfully performing risk analysis at a high level requires a combination of technical knowledge and specialized analytical skills:

- **Analytical Ability:** Strong analytical and problem-solving skills are essential for effective risk assessment.

- **Strategic Planning:** The ability to translate risk findings into actionable, long-term strategic plans is a key requirement for risk managers and leaders.
- **Framework Knowledge:** Professionals must have in-depth knowledge of cybersecurity frameworks (e.g., ISO 27001) and risk management best practices.

I'm a computer science major, please generate 5 backend, and 5 cyber security project I can do in about 3 days

Given your background as a computer science major and your three-day timeframe, here are five backend and five cybersecurity projects selected from the sources that are categorized as "Easy" to "Moderate" in difficulty.

5 Backend Projects

These projects focus on building core server-side skills, including CRUD operations, database management, and external API integration.

1. **Task Manager API**: Build a simple RESTful API to manage user tasks. This project involves implementing **JWT Authentication** for secure login and handling **CRUD operations** (Create, Read, Update, Delete) using a database like SQLite or PostgreSQL.
2. **Weather API Wrapper Service**: Create a service that fetches real-time data from an external provider like OpenWeatherMap. To increase complexity for a three-day window, you can add a **caching strategy** using Redis to reduce the number of external API calls.
3. **URL Shortener**: Develop a service that takes long URLs and generates a unique short link. This requires setting up a database to store URL pairs and configuring **HTTP redirects** so users are seamlessly sent to their destination.
4. **Expense Tracker API**: Build an API that allows users to sign up, log in, and manage their spending. This project emphasizes **data modeling** (categorizing expenses like groceries or health) and protecting endpoints with JWTs.
5. **Notes Application API**: Use a framework like **Django** or **FastAPI** to build a secure notes app. Users should be able to create, edit, and tag notes, with all data stored in a relational database like PostgreSQL.

5 Cybersecurity Projects

These projects focus on hands-on technical skills, ranging from writing your own tools in Python to setting up defensive labs.

1. **Python-Based Port Scanner**: Write a script that takes an IP address and scans it to discover open ports and services. Using Python's `socket` library, you can build a tool that identifies which software versions are running on a target.
2. **Keylogger Development**: Code a basic program in Python (under 10 lines) that captures keystrokes on a target machine and saves them to a log file. This project helps you understand how malware can harvest sensitive information like passwords.
3. **Vulnerable Web App Lab (DVWA)**: Set up a virtual machine running the **Damn Vulnerable Web Application (DVWA)**. Practice offensive techniques like **SQL injection** or **command injection** in a controlled "low security" environment.
4. **Phishing Attack Simulator**: Use an open-source framework like **Gophish** to set up a simulated phishing campaign. You will learn how to create email templates, launch scheduled campaigns, and track results in real-time.

5. **System Hardening and Log Analysis**: Start with a standard Windows 10/11 VM and run an `nmap` scan to see exposed services. Perform **hardening** (disabling unused services, turning on firewalls), and then run a second scan to compare logs and verify your improved security posture.

Timeline Tip
To complete these in three days, the sources suggest focusing on a **small, mission-driven setup**. Rather than trying to build a massive infrastructure, aim for a lab with 2–4 virtual machines (like Kali Linux and a vulnerable target) that you can break and rebuild quickly.
generate 5 more beginner cyber security project that can be done in 3 days
Based on the sources, here are five additional beginner cybersecurity projects for a computer science major that can be completed in approximately three days:

1. Build a Simple Backdoor in Python
Using your coding skills, you can create a basic **backdoor**—a hidden access point that bypasses normal authentication.

- **The Task:** Use Python's `socket` library to build a two-part program: a **server** that waits for a connection and a **client/backdoor** that initiates it.
- **Learning Goal:** Understand how attackers maintain persistent access. You can extend the project using the `subprocess` library to **execute commands** on the target system remotely.

2. Set Up a Host-based Intrusion Detection System (HIDS)
This project focuses on **defensive security** by monitoring a specific device for suspicious activity.

- **The Task:** Install **OSSEC**, an open-source HIDS, on a Linux VM. You will use the **command line** to configure the tool for **log analysis, integrity checking, and rootkit detection**.
- **Learning Goal:** Gain experience with real-time alerts and understanding how to navigate security tools without a GUI.

3. Build a SOC Home Lab with SIEM
A **Security Operations Center (SOC) lab** allows you to act as a "digital security guard" for a network.

- **The Task:** Use a platform like **Elastic** to build a lab that aggregates logs from different sources. You can generate "attacks" from a Kali Linux VM and watch how they appear as **alerts in your SIEM dashboard**.
- **Learning Goal:** Practice **alert triage**—the process of determining if an event is a "True Positive" (real threat) or a "False Positive".

4. Secure Azure Active Directory (Microsoft Entra ID)
As organizations move to the cloud, **Identity and Access Management (IAM)** has become a critical skill.

- **The Task:** Set up a trial Azure environment to manage users and groups in a **hybrid identity solution**. Focus on configuring **Multi-Factor Authentication (MFA)** and managing **administrative units**.

- **Learning Goal:** Understand how to protect cloud infrastructure and ensure only trusted users have access to sensitive data.

5. Network Vulnerability Assessment with Nessus
Professional penetration testers often begin an engagement with a **scanning stage** to identify "weak links".

- **The Task:** Install a vulnerability scanner like **Nessus** in your home lab. Run a scan against a target, such as a **Metasploitable VM**, to identify out-of-date operating systems and misconfigured services.
- **Learning Goal:** Learn to interpret **vulnerability reports** and prioritize which flaws represent the highest risk to an organization's "crown jewels".

tell me more about the key logger development project, how would i start, what would the timeline for completing this project in 3 days look like
A **keylogger** is a type of spyware that silently captures and stores each keystroke typed on a keyboard. In a cybersecurity context, these projects are designed to help you understand how attackers harvest sensitive information, such as **usernames, passwords, and banking details**, from an infected device. Because this type of software can be used for malicious purposes, you must only test it on systems you own or have explicit permission to use to avoid legal consequences.
How to Start
As a computer science major, you can approach this project using two distinct methods:

1. **Code-Based (Python)**: This is the most effective way to demonstrate your technical skills. You will need to install the `keyboard` library (`sudo pip3 install keyboard`). A basic Python keylogger can be written in **under 10 lines of code**. It works by capturing keystrokes on the machine where it is running and saving them to a local file, typically named `keystrokes.log`. Note that running this script usually requires **administrator or root privileges**.
2. **Tool-Based (Kali Linux)**: This method involves using the **Metasploit Framework**. You use `msfvenom` to create a **Meterpreter shell** payload (an `.exe` file). Once the payload is executed on a target Windows machine, you use `msfconsole` on your Kali machine to set up a listener. Once a connection is established, you can use built-in commands like `keyscan_start` to begin logging and `keyscan_dump` to view the captured data.

3-Day Project Timeline
To complete this project within your three-day window, focus on a small, mission-driven setup in a **virtual home lab** to ensure the "attack" remains isolated from your actual network.

- **Day 1: Virtual Lab Setup**:
  - Install a hypervisor like **Oracle VirtualBox** or **VMware Workstation Player**.
  - Set up two virtual machines (VMs): **Kali Linux** (Attacker) and **Windows 10/11** (Target).
  - Configure a **Host-only network** so the VMs can communicate with each other but remain fenced off from the internet and your home network.
- **Day 2: Implementation and Testing**:

    - **Python Track**: Write your script using the `keyboard` library. Test it locally on your Windows VM and verify the data is being saved to `keystrokes.log`.
    - **Kali Track**: Generate your payload with `msfvenom`. Disable Antivirus on the target Windows VM (to simulate a successful bypass) and execute the file.
    - Take **snapshots** of your VMs before execution so you can quickly "rebuild" and revert your lab to a clean state.
- **Day 3: Advanced Features and Documentation**:
    - **Upgrading the Code**: Try to enhance your Python script by adding a function to **send the logged data** over a network socket to a third-party listener device.
    - **Verification**: If you have a defensive VM like a SIEM or HIDS, check the logs to see if your keylogger's activity was detected.
    - **Documentation**: Record your findings, including how you configured the network and the specific commands you used, to turn this into a professional **portfolio asset**.

tell me more about building a simple backdoor in python, who can it be completed in 3 days
A **backdoor** is a hidden, unauthorized access point in a computer system or network that allows an attacker to bypass normal authentication and maintain persistent access. In a Python-based project, you can build a simple version of this by creating a two-part program: a **server** (the attacker's machine) and a **client/backdoor** (the target machine).
Building a Backdoor in Python: The Basics
The core of a Python backdoor relies on establishing a network connection between two devices.

- **The Library**: You use the socket **library** to handle the communication. The server waits for a connection, and the client initiates it.
- **The Connection**: You must ensure that the IP address and port number match in both the server and client scripts for them to communicate successfully.
- **Adding Power**: To make the backdoor functional, you can integrate other libraries:
    - subprocess: This allows you to execute system-level commands on the target machine remotely.
    - os: This can be used to navigate and change directories on the target system.
    - threading: This allows the server to maintain multiple connections at once if you are scanning a broader lab network.

3-Day Project Timeline
To complete this safely and effectively, you should use a **virtual home lab** setup to ensure the "attack" does not leak onto your actual home network.
Day 1: Lab Infrastructure & Safety

- **Hypervisor Setup**: Install a hypervisor like **Oracle VirtualBox** or **VMware Workstation Player**.
- **VM Deployment**: Set up two virtual machines: **Kali Linux** (as your Attacker/Server) and a target VM like **Windows 10** or **Ubuntu** (as your Victim/Client).
- **Network Isolation**: Configure a **Host-only network** (e.g., `192.168.56.0/24`) so the VMs can talk to each other but are fenced off from the internet.

- **Snapshots**: Take **Baseline snapshots** of both VMs so you can quickly reset them if you break something.

Day 2: Core Script Development

- **Coding the Socket**: Write your `server.py` and `client.py` scripts.
- **Testing Connectivity**: Run the server on Kali and the client on the target VM. Verify that the client sends a successful "connection established" message to the server.
- **Troubleshooting**: If the connection fails, check the virtual network settings and ensure no guest-level firewalls are blocking the specific port you chose.

Day 3: Feature Integration & Execution

- **Remote Command Execution**: Integrate the `subprocess` library into your client script so it can receive commands from the server and return the results.
- **Executing the Payload**: "Infect" the target VM by running your client script. Use the server in Kali to run commands like `whoami`, `ipconfig`, or `dir` to prove you have access.
- **Final Documentation**: Take screenshots of the command results and document your network configuration to create a professional portfolio piece.

Safety Note: Building and testing backdoors should only be done on systems you own or have explicit permission to use, as unauthorized use can lead to serious legal consequences.
give me a list of 10 more simple cyber security projects that can be done in 3 days
Based on the sources, here are 10 additional beginner cybersecurity projects for a computer science major that can be completed in approximately three days:

1. OSINT Investigation Lab

Open-Source Intelligence (OSINT) involves gathering publicly available data to build a profile of a target.

- **The Task:** Use professional networking sites and specialized tools to extract a list of names, emails, job titles, and locations for a "mock" target company.
- **Goal:** Understand how attackers conduct reconnaissance before a red team engagement.

2. Digital Forensics Artifact Analysis

Digital forensics experts examine **digital artifacts** (traces left behind in systems) to understand security incidents.

- **The Task:** Perform common actions on a Windows VM (like browsing specific sites or creating files), then use forensic tools to locate and analyze the "traces" these actions left in the registry or memory.
- **Goal:** Learn how to collect and preserve digital evidence for investigation.

3. Basic Honeypot Deployment

A **honeypot** is a computer intentionally left vulnerable to catch and identify attacker techniques.

- **The Task:** Set up a simple honeypot listener in your virtual home lab. Use your Kali Linux VM to "attack" it and observe how the honeypot logs the interaction.
- **Goal:** Gain insight into how attackers probe systems and how to use deceptive defenses.

4. Wireshark Network Packet Analysis
Analyzing network traffic is essential for detecting unencrypted data transfers.

- **The Task:** Use **Wireshark** to capture traffic between two VMs in your lab. Simulate a login to a service using an unencrypted protocol (like HTTP) and find the captured username and password in the packets.
- **Goal:** Understand the importance of encryption and how "eavesdropping" attacks function.

5. Secure Password Generator and Validator
Coding your own tools is a great way to demonstrate technical proficiency.

- **The Task:** Write a Python script that generates random, secure passwords based on length and special character requirements. Extend it to check common "weak password" lists to see if a chosen password is too simple.
- **Goal:** Learn the logic behind password complexity and brute-force protection.

6. Default Credential Auditing
Many devices are compromised because they still use "out-of-the-box" settings.

- **The Task:** Research common default credentials for lab targets (like an MFP printer or a router) and test them against your VMs to see if they grant unauthorized access.
- **Goal:** Practice identifying "low-hanging fruit" vulnerabilities during a security audit.

7. Cybersecurity Job Simulation (Forage)
Job simulations provide a taste of real-world tasks performed by professionals at major firms.

- **The Task:** Complete a free simulation from companies like **MasterCard or PWC** on the Forage platform. These typically involve tasks like security analysis or red teaming.
- **Goal:** Experience the "day-to-day" workflow of a professional security analyst.

8. IT Ticketing and Incident Response Simulation
Security roles often involve handling user-generated requests via ticketing systems.

- **The Task:** Use a simulation (like CourseCareers) to practice responding to common security-related tickets, such as password resets, troubleshooting suspicious software, or escalating a potential breach.
- **Goal:** Develop the communication and documentation skills required for incident response.

9. Linux Server Hardening
Defending a system requires more than just installing a firewall; it requires active **hardening**.

- **The Task:** Start with a standard Linux VM (like Ubuntu). Implement security measures such as disabling root login, configuring SSH keys, and setting up a basic firewall like **UFW**.
- **Goal:** Learn to reduce the "attack surface" of a server.

10. Vulnerability Scanning with Open-Source Tools
Beyond basic port scanning, you can use specialized tools to find deeper flaws.

- **The Task:** Install and run an open-source vulnerability scanner in your home lab to identify insecure software configurations or out-of-date systems on your target VMs.

- **Goal:** Practice the cycle of identifying, prioritizing, and mitigating vulnerabilities.

tell me more about the honeypot deployment project how long should it take
A **honeypot** is a computer system designed to be **intentionally left vulnerable** to attract attackers. In a cybersecurity lab context, its purpose is to "catch" unauthorized users and identify the specific techniques, tools, and exploits they use. By observing these interactions in a controlled environment, you can gain a deeper understanding of real-world threats without risking your actual data or systems.
Core Components of the Project
To build a functional honeypot deployment within your 3-day window, the sources suggest focusing on a **small, mission-driven virtual setup**.

- **The Victim (The Honeypot):** Use a pre-configured vulnerable VM like **Metasploitable 2**, which is designed to be full of holes and easily compromised for learning purposes.
- **The Network (Isolation):** This is the most critical step. The honeypot must reside on an **isolated "host-only" network** (e.g., `192.168.56.x`). This ensures that your intentionally vulnerable machine never touches your real home network or the public internet, keeping the "attack" strictly on the virtual playmat.
- **The Attacker:** Use a **Kali Linux VM** to act as the adversary, running scans and exploits against your honeypot to generate data for analysis.

3-Day Project Timeline
While a basic lab can be stood up in just a few hours, a three-day timeline allows you to move beyond simple installation to actual analysis and documentation.

- **Day 1: Infrastructure and Safety**
  - Install a hypervisor like **Oracle VirtualBox** and enable virtualization in your BIOS.
  - Configure a **Host-only network** to create a secure, fenced-off subnet for your lab.
  - Take **baseline snapshots** of your host and empty VMs so you can quickly rebuild if something breaks.
- **Day 2: Deployment and Verification**
  - Import the **Metasploitable 2 image** and install your **Kali Linux attacker machine**.
  - "Wire" both VMs to your isolated host-only adapter.
  - **Verify isolation:** Confirm that the honeypot can talk to Kali but **cannot** browse the internet or see other devices on your physical router.
- **Day 3: Interaction and Analysis**
  - From Kali, run an **Nmap scan** to discover the "open" services you've intentionally left exposed on the honeypot.
  - Simulate an attack (e.g., attempting to log in with default credentials) and observe how the system logs the event.
  - **Documentation:** Record the problem you solved and the logs you captured; this turns a simple lab into a **career asset** that you can show to hiring managers.

can you give me more details on the keylogger project

A **keylogger** is a program designed to capture and store every keystroke typed by a user on a keyboard. In a cybersecurity learning context, this project helps you understand how attackers harvest sensitive information, such as **usernames, passwords, bank details, and social media updates**.
Because this software can be used for malicious purposes and may constitute wiretapping, you must obtain explicit permission from a computer's owner **before** testing it on any device that is not your own.

Two Methods for Implementation

The sources outline two primary ways to approach this project, depending on whether you want to focus on using existing tools or writing your own code.

1. Python-Based Development (Code-Focused)

This method is recommended for a computer science major to demonstrate technical coding skills.

- **How it Works**: A basic script (under 10 lines of code) captures keystrokes on the machine where it is running and saves them to a local file, typically named `keystrokes.log`.
- **Requirements**: You need the keyboard **library**, which can be installed via `sudo pip3 install keyboard`.
- **Execution**: Running the script usually requires **administrator or root privileges** to access the keyboard input directly.
- **Upgrading**: You can enhance this by adding a function to send the captured data over a network socket to a third-party listener device.

2. Kali Linux and Metasploit (Tool-Focused)

This method uses the **Metasploit Framework** to understand how keyloggers are deployed in real-world "backdoor" scenarios.

- **The Payload**: Use `msfvenom` to create a **Meterpreter shell** payload (e.g., `shell.exe`). You configure the LHOST (your Kali IP) and LPORT (the port to connect back to).
- **The Listener**: Run `msfconsole` on your attacker machine to set up a listener that waits for the "infected" target to connect.
- **Commands**: Once a connection is established, use built-in Meterpreter commands:
  - `keyscan_start`: Begins capturing keystrokes on the target Windows machine.
  - `keyscan_dump`: Outputs the captured keystrokes to your terminal.

Suggested 3-Day Project Timeline

To complete this safely within three days, use a **virtual home lab** (such as VirtualBox or VMware) with isolated "host-only" networking to ensure no traffic leaks to your physical network.

- **Day 1: Infrastructure and Safety Setup**
  - Install a hypervisor and set up two virtual machines: **Kali Linux** (Attacker) and a **Windows 10/11 Eval** (Target).

    - Configure a **Host-only network** (e.g., `192.168.56.x`) so the VMs can communicate only with each other.
    - Take **Baseline snapshots** of both VMs so you can quickly reset them if needed.
- **Day 2: Core Development and Testing**
    - **Python Track**: Write the script using the `keyboard` library and verify it logs to a file locally on the Windows VM.
    - **Kali Track**: Generate the payload with `msfvenom`. Disable Windows Defender on the target VM (to simulate a successful bypass) and execute the payload to establish a Meterpreter shell.
    - Practice starting and dumping keystrokes to see the data captured.
- **Day 3: Advanced Features and Documentation**
    - Attempt to upgrade your Python script to send data over a network socket to a simple server script on your Kali machine.
    - **Verification**: If you have a defensive tool like a HIDS installed, check to see if the keylogger activity was detected.
    - **Document Your Work**: Record the network configuration, the commands used, and screenshots of the logs to turn this into a professional portfolio asset.

what kind of work would go into this porject
Based on our recent discussion, the work required to complete the **keylogger project** within a three-day window involves a mix of cybersecurity ethics, infrastructure setup, offensive tool configuration, and Python programming.

1. Ethical and Legal Compliance

The first step is ensuring you have **explicit permission** from the owner of the target device. Unauthorized use of a keylogger is considered **wiretapping** and can carry serious legal consequences. In a learning context, this work is typically done entirely within a self-owned virtual lab.

2. Infrastructure Setup (The "Playmat")

You will need to build an isolated virtual environment to keep your experiments safe:

- **Hypervisor Installation:** Installing software like Oracle VirtualBox to run multiple virtual machines (VMs).
- **Network Isolation:** Configuring a **"Host-only" network** so your attacker machine (Kali Linux) can talk to your target machine (Windows) without any traffic leaking to the internet or your home network.
- **Snapshot Management:** Taking "Baseline" snapshots of your VMs before starting so you can instantly revert to a clean state if something breaks.

3. Technical Implementation (Two Tracks)

You can approach the project using two different methods to broaden your skillset:

The Tool-Based Track (Kali Linux):

- **Payload Creation:** Using `msfvenom` to generate a **Meterpreter shell** (.exe file). This requires you to specify your local IP (LHOST) and a chosen port (LPORT) for the connection.

- **Listener Configuration:** Opening `msfconsole` to set up a listener that waits for the target machine to connect back to you.
- **Bypassing Defenses:** Temporarily disabling antivirus software on the target system to allow the payload to execute.
- **Active Logging:** Running specific commands like `keyscan_start` to begin recording and `keyscan_dump` to output the results to your terminal.

The Code-Based Track (Python):

- **Library Integration:** Installing and importing the `keyboard` library.
- **Script Logic:** Writing a program (often **under 10 lines of code**) that listens for keyboard events and writes them to a file.
- **File Management:** Handling the creation and updating of a `keystrokes.log` file on the target system.
- **Permission Handling:** Running the script with **administrator or root privileges**, which is required to access the keyboard's input stream.

4. Analysis and Portfolio Development

The final stage involves verifying your results and documenting your process:

- **Log Review:** Using commands like `cat` to view the captured data and confirm that special keys (like Backspace) were recorded correctly.
- **Documentation:** Recording your network architecture, the commands used, and the problems you solved. This transforms a simple exercise into a **career asset** for your portfolio that proves your technical proficiency to employers.

Today • 12:29 PM

21 sources

Studio

Audio Overview

Video Overview

Mind Map

Reports

Flashcards

Quiz

Data Table

From Pipeline Hacks to Hallway Printers
Deep Dive · 10 sources · 2d ago
Hacking Your Career
10 sources · 2d ago
Cybersecurity Flashcards
20 sources · 2d ago
The Cybersecurity Career Blueprint: From Lab to Leadership 10 sources · 2d ago

NotebookLM can be inaccurate; please double check its responses.